

IFT3335 - Introduction à l'intelligence artificielle

Devoir 1

Aide à la Navigation : Exploration des algorithmes de recherche

Prof. Jian-Yun Nie, Maxime Gauthier

Date de remise : 12 octobre

1 Informations administratives

- Le devoir compte pour 15% de la note finale.
- Le devoir doit être complété en équipe de 2 à 3 personnes.
- Indiquer le **Prénom**, **Nom** et **Matricule** de chaque membre de l'équipe dans les fichiers `src/student_solver.py` et `questions_theoriques.txt`.
- Le travail comporte deux sections : une section **pratique** et une section **théorique**.
- La remise s'effectue sur Studium. Vous devez remettre uniquement les deux fichiers que vous avez modifiés, soit `src/student_solver.py` et `questions_theoriques.txt`.
- La remise sur Studium doit être faite avant le **12 octobre 23h55**. Un seul membre de l'équipe remet le devoir.

2 Mise en contexte

Les applications d'assistant à la navigation comme Google Maps ou Waze ont comme objectif de donner le trajet le plus court (en terme de temps) entre une destination de départ et d'arrivée. Le cœur du bon fonctionnement de ces applications repose sur l'algorithme de recherche pour identifier le chemin le plus court.

Dans ce devoir, on explorera et testera différents algorithmes de recherche pour trouver le chemin le plus court en terme de distance (et non de temps comme les applications classiques), et ce de manière la plus optimale possible. Trouver le chemin le plus rapidement possible est essentiel pour avoir une bonne expérience utilisateur.

L'objectif du devoir est de comparer les différents algorithmes de recherche qui tentent de trouver le chemin de plus courte distance le plus efficacement possible (avec le moins de nœuds explorés) entre **START** et **GOAL** sur cette carte du Plateau Mont-Royal. On compte 647 intersections de rues.



FIGURE 1 – Carte du Plateau Mont-Royal avec le point de départ (START) et d'arrivée (GOAL).

Cette carte peut être abstraite et modélisée comme un graphe **orienté** pondéré où les sommets du graphe (les nœuds) sont les intersections de rues, les arêtes sont les rues et le poids des arêtes est la longueur (en mètres) entre deux intersections. Le graphe est orienté parce que les sens uniques sont respectés : une arête de u vers v n'implique pas qu'il existe une arête de v vers u .



FIGURE 2 – Abstraction de la carte du Plateau Mont-Royal sous forme de graphe.

Dans le code, le graphe est représenté par une liste d'adjacence pondérée nommée `adj`. Concrètement, `adj` est un **dictionnaire** de type `dict[int, list[tuple[int, float]]]` qui contient 647 clés, une par nœud. Chaque nœud est identifié par un entier entre 0 et 646. La valeur associée à la

clé `u` est la liste des voisins de `u`, c'est-à-dire des nœuds que l'on peut atteindre directement depuis `u`. Chaque voisin est représenté par un tuple (`ID`, `Distance`), où `ID` est l'identifiant du voisin (`int`) et `Distance` est la longueur de la rue qui relie `u` à ce voisin, en mètres (`float`). Dans les signatures de fonctions, ce type porte le nom `Adjacency`. Il s'agit d'un simple alias de type, pas d'une classe : `adj` se manipule comme n'importe quel dictionnaire Python.

Ainsi, voici quelques exemples pour manipuler le graphe :

```
adj[9] -> [(11, 21.0), (15, 34.4), (24, 9.2)] # Liste des noeuds
        voisins au noeud 9
adj[9][0] -> (11, 21.0) # Le tuple du noeud
        voisin 11
adj[9][0][1] -> 21.0 # Le poids (distance) de
        l'arete, la distance entre le noeud 9 et 11
```

3 Explication du travail

3.1 Section pratique (code)

Le travail se compose de deux parties. La première est l'implémentation de quatre différents algorithmes de recherches pour observer, comparer et établir lequel ou lesquels sont les plus appropriés pour déterminer le chemin de plus courte distance entre deux destinations le plus efficacement possible.

Voici les étapes pour être prêt à travailler sur le devoir :

1. Télécharger le dossier zip sur studium dans l'onglet TP1.
2. Unzip le dossier et ouvrez ce dernier dans votre IDE préféré.
3. Ouvrez le terminal et entrer les commandes suivantes :

```
# Windows (PowerShell)
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

4. Vous êtes prêt à travailler dans le fichier `src/student_solver.py`.

3.2 Section théorique

Quelques questions théoriques **évaluées** se trouvent dans le fichier `questions_theoriques.txt` qui ne nécessite aucune implémentation. Ce sont uniquement des questions pour pousser la réflexion et évaluer la compréhension du contexte et des adaptations à ce dernier.

4 Travail à réaliser

4.1 Section pratique (code - 80 pts)

Votre travail consiste à remplir les fonctions marquées `TODO` dans le fichier `src/student_solver.py` ainsi que répondre aux questions théoriques qui se trouvent dans le fichier `questions_theoriques.txt`. **Les autres fichiers ne doivent absolument pas être modifiés.** Vous pouvez les consulter pour avoir une compréhension plus globale du travail, mais vous devriez être en mesure de bien comprendre et compléter le travail uniquement avec le fichier `src/student_solver.py`.

Voici les algorithmes de recherche que vous devez implémenter dans le fichier `src/student_solver.py`. Quelques consignes globales à suivre pour l'implémentation de chaque fonction :

- Ne changez pas la signature des fonctions. Gardez le nom de la fonction, son type de retour, ses paramètres ainsi que leur type associé.
- Les variables locales définies doivent obligatoirement être utilisées et peuplées. Vous pouvez ajouter d'autres variables locales que vous jugez nécessaires à l'implémentation.
- Ne supprimez pas de code déjà écrit, vous n'avez qu'à ajouter du code pour compléter entièrement le devoir. **Seule exception** : dans les deux fonctions d'heuristique, remplacez la ligne `return 0.0` par votre implémentation.

Les signatures utilisent trois types déjà importés en tête de `src/student_solver.py`. Vous n'avez pas besoin de consulter leur définition : voici tout ce qu'il faut savoir pour les utiliser.

- **Adjacency** (paramètre `adj`) : le graphe, sous forme de dictionnaire `{nœud: [(voisin, distance), ...]}` (voir la section 2). Les voisins de `u` s'obtiennent avec `adj[u]`. Chaque voisin est un tuple qu'on peut décomposer directement dans une boucle : `for voisin, distance in adj[u]:`. Les distances sont en mètres.
- **CoordsM** (paramètre `coords_m`, heuristiques seulement) : un dictionnaire `{nœud: (x, y)}` qui donne la position de chaque nœud en **mètres**, `x` vers l'est et `y` vers le nord. Pour lire la position d'un nœud : `x, y = coords_m[node]`.
- **SearchResult** (valeur de retour des quatre algorithmes) : un objet qui regroupe le résultat d'une recherche. Il se crée en passant ses quatre valeurs dans cet ordre :

```
SearchResult(chemin, cout, expanded, n_explored)
```

- `chemin` : liste des nœuds de `start` à `goal` (inclus), ou `None` si aucun chemin n'existe ;
- `cout` : longueur du chemin en mètres, ou `math.inf` si aucun chemin n'existe ;
- `expanded` et `n_explored` : les variables déjà déclarées dans chaque fonction.

Vous n'avez qu'à créer cet objet à la fin de votre algorithme : vous n'aurez jamais à lire ses attributs.

1. Parcours en Largeur (BFS) (12 points)

```
def bfs(adj: Adjacency, start: int, goal: int) -> SearchResult:
```

Vous devez implémenter l'algorithme de recherche non informé de parcours en largeur tel que vu en cours.

2. Parcours en Profondeur (DFS) (12 points)

```
def dfs(adj: Adjacency, start: int, goal: int) -> SearchResult:
```

Vous devez implémenter l'algorithme de recherche parcours en profondeur tel que vu en cours.

3. Recherche à Coût Uniforme (UCS) (16 points)

```
def ucs(adj: Adjacency, start: int, goal: int) -> SearchResult:
```

Vous devez implémenter l'algorithme de recherche à coût uniforme UCS qui explore les noeuds en fonction du coût total du chemin parcouru depuis le noeud initial.

4. Heuristique Euclidienne (6 points)

```
def heuristic_euclidean(node: int, goal: int, coords_m: CoordsM)
    -> float:
```

Vous devez implémenter l'heuristique euclidienne pour calculer la distance entre le nœud courant et le nœud d'arrivée. La distance euclidienne est la distance « à vol d'oiseau » entre deux points dans un espace à deux dimensions. Elle est définie par l'équation suivante :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (1)$$

où (x_1, y_1) sont les coordonnées de **node** et (x_2, y_2) celles de **goal**.

5. Heuristique Manhattan (6 points)

```
def heuristic_manhattan(node: int, goal: int, coords_m: CoordsM)
    -> float:
```

Vous devez implémenter l'heuristique de Manhattan pour calculer la distance entre le nœud courant et le nœud d'arrivée. La distance de Manhattan mesure la distance entre deux points en additionnant les différences absolues de leurs coordonnées. Elle est définie par la formule suivante :

$$d = |x_1 - x_2| + |y_1 - y_2| \quad (2)$$

6. Recherche A* (28 points)

```
def a_star(adj: Adjacency, start: int, goal: int, heuristic) ->
    SearchResult:
```

Vous devez implémenter l'algorithme de recherche A*, $f(n) = g(n) + h(n)$.

Utilisation des paramètres et des variables.

- **adj**, **start**, **goal**, **came_from**, **expanded** et **n_explored** : identiques à UCS.
- **heuristic** est une **fonction déjà prête** qui prend un seul argument, un nœud, et renvoie $h(n)$ en mètres : **h = heuristic(v)**. Ne lui passez ni **goal** ni **coords_m**, car c'est déjà fait. N'appellez pas non plus directement **heuristic_euclidean** ou **heuristic_manhattan**

dans `a_star` : c'est `main.py` qui choisit l'heuristique transmise. Votre A* doit donc fonctionner avec n'importe quelle heuristique. Assurez vous de renvoyer le coût réel $g(goal)$, et non $f(goal)$

4.2 Tester votre code

Voici les commandes à entrer dans le terminal pour tester l'implémentation de chacune de vos fonctions.

Pour tester indépendamment chaque algorithme sans visualisation :

```
python src/main.py --algo bfs --no-plot
python src/main.py --algo dfs --no-plot
python src/main.py --algo ucs --no-plot
python src/main.py --algo astar --no-plot      # Heuristique
Euclidienne
```

Pour comparer et visualiser les différents algorithmes de recherche tous en même temps :

```
python src/main.py --algo tous --no-plot
python src/main.py --algo tous              # Pour voir la
visualisation
```

Pour visualiser les algorithmes sur la carte :

```
python src/main.py --algo bfs
python src/main.py --algo dfs
python src/main.py --algo ucs
python src/main.py --algo astar
python src/main.py --algo astar --heuristique manhattan
```

4.3 Section Théorique

Répondez à ces questions dans le fichier `questions_theoriques.txt`. Les questions y sont déjà écrites, écrivez votre réponse sous la question dans l'espace réservé.

1. **(6 points)** Comparez les résultats obtenus, soit le nombre de nœuds explorés, le chemin obtenu ainsi que la valeur de ce dernier pour l'algorithme A* avec les heuristiques de Manhattan et Euclidienne. Est-ce que les deux sont identiques ou il y en a une meilleure que l'autre ?
2. **(4 points)** Lequel ou lesquels des algorithmes trouvent le chemin de plus courte distance entre le point de départ et le point d'arrivée, c'est-à-dire, qui trouvent le chemin de distance minimum entre le point de départ et d'arrivée ?
3. **(4 points)** Lequel ou lesquels des algorithmes choisiriez-vous pour votre future application d'aide à la navigation ? Basez votre choix sur la fiabilité ainsi que la rapidité de la réponse trouvée.
4. **(6 points)** Le contexte établi dans le devoir est une carte du plateau Mont Royal qui s'étend sur environ 4 km × 4 km et contient 647 nœuds. Si la carte change pour une carte du Canada de Montréal à Vancouver, le nombre d'intersections (nœuds) augmenterait, quel problème

pourrions-nous rencontrer dans les algorithmes implémentés dans ce devoir ? Proposez un autre algorithme ou une variante pour régler ce problème.

5 Correction et barème

Le devoir est noté sur 100 points : 80 points pour la section pratique et 20 points pour la section théorique. La section pratique est corrigée automatiquement par des tests unitaires. Pour la section théorique, une attention particulière sera portée à la clarté et à la justification de vos réponses.

Élément	Description	Points
<i>Section pratique</i>		
<code>bfs</code>	Parcours en largeur	12
<code>dfs</code>	Parcours en profondeur	12
<code>ucs</code>	Recherche à coût uniforme	16
<code>heuristic_euclidean</code>	Heuristique euclidienne	6
<code>heuristic_manhattan</code>	Heuristique de Manhattan	6
<code>a_star</code>	Recherche A*	28
<i>Section théorique</i>		
Question 1	Manhattan contre euclidienne	6
Question 2	Algorithmes optimaux	4
Question 3	Choix pour une application de navigation	4
Question 4	Passage à l'échelle	6
Total		100

TABLE 1 – Barème détaillé du devoir.